



Debugging Windows Programs

To successfully identify and exploit buffer overflows in Windows programs, we need to debug the program to follow its execution flow and its data in memory. There are many tools we can use for debugging, like [Immunity Debugger](#), [OllyDBG](#), [WinGDB](#), or [IDA Pro](#). However, these debuggers are either outdated (Immunity/OllyDBG) or need a pro license to use (WinGDB/IDA).

In this module, we will be using [x64dbg](#), which is an excellent Windows debugger aimed specifically at binary exploitation and reverse engineering. [x64dbg](#) is an open-source tool developed and maintained by the community and also supports x64 debugging (unlike Immunity), so we can keep using it when we want to move to Windows x64 buffer overflows.

In addition to the debugger itself, we will utilize a binary exploitation plugin to efficiently carry out many tasks required for identifying and exploiting buffer overflows. One popular plugin is [mona.py](#), which is an excellent binary exploitation plugin, though it is no longer maintained, does not support x64, and runs on Python2 instead of Python3.

So instead, we will be using [ERC.Xdbg](#), which is an open-source binary exploitation plugin for x64dbg.

Installation

All of these tools are already installed on the Windows VM found at the end of the section, which you can connect to from the Pwnbox with the below command:

```
shellsession
kimvutrinh@htb[/htb]$ xfreerdp /v:<target IP address> /u:htb-student /p:<password>
```

You can also use the same command on your own Linux VM or connect to the Windows VM with RDP on Windows or macOS. To connect to the VM from your machine, you must first connect using the VPN key found at the end of the section. It is also possible to install the tools on your own Windows VM, as shown next.

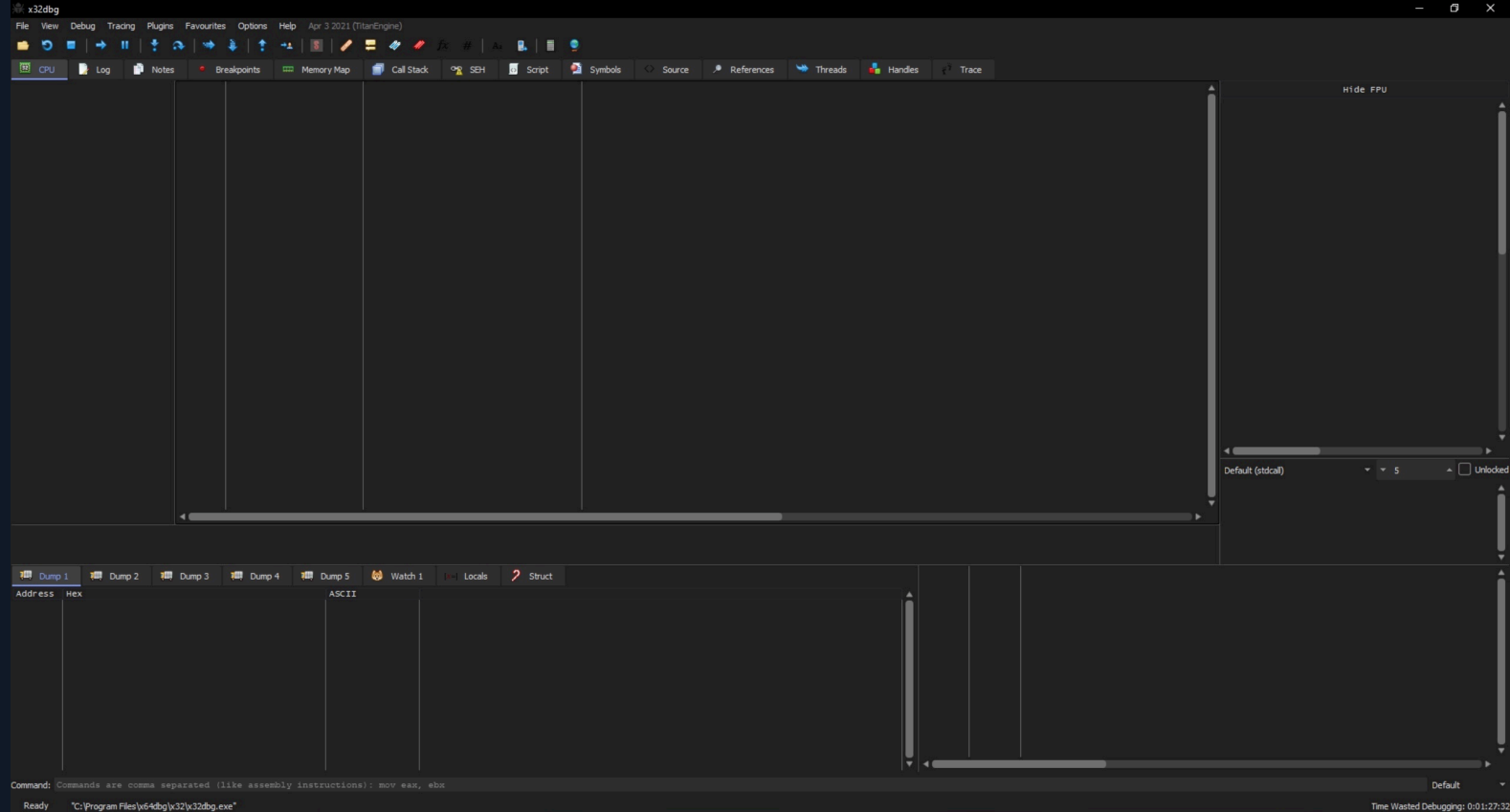
x64dbg

To install `x64dbg`, we can follow the instructions as shown in its [GitHub Page](#), and go to the [latest release](#) page, and download the `snapshot_<SNIP>.zip` file. Once we download it in our Windows VM, we can extract the `zip` archive content, rename the `release` folder to something like `x64dbg`, and move it to our `C:\Program Files` folder, or keep it in any folder we want.

Finally, we can double-click on `C:\Program Files\x64dbg\x96dbg.exe` to register the shell extension and add a shortcut to our Desktop.

Note: `x64dbg` comes with two separate applications, one for `x32` and one for `x64`, each under their folder. Clicking on `x96dbg.exe` as noted above will register the version that matches our Windows VM, which in our case is the `x32` one.

Once that's done, we can find the `x32dbg` icon on our Desktop, and we can double-click it to start our debugger:



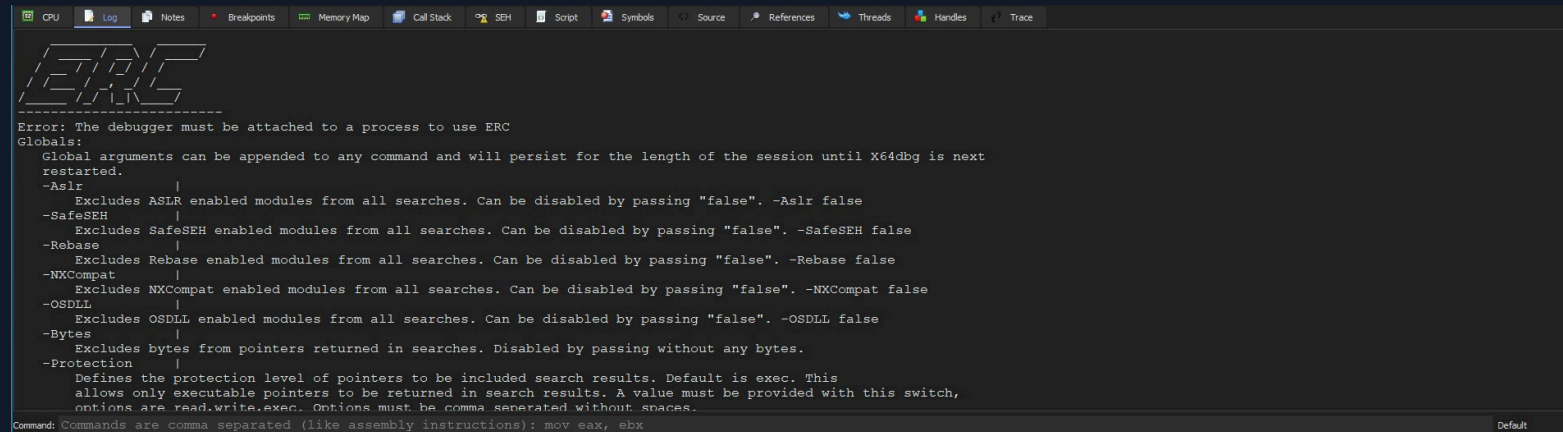
Tip: To use the dark theme like the above screenshot, simply go to **Options > Theme** and select **dark** .

ERC

To install the **ERC** plugin, we can go to the [release page](#), and download the **zip** archive that matches our VM (**x64** or **x32**), which in our case is **ERC.Xdbg_32-<SNIP>.zip** . Once we download it to our Windows VM, we can extract its content into **x32dbg** plugins folder located in **C:\Program Files\64dbg\32\plugins** .

When that's complete, the plugin should be ready for use. So, once we run **x32dbg** , we can type **ERC --help** in the command bar at the bottom to view **ERC** 's help menu.

To view the `ERC` 's output, we must switch to the `Log` tab by clicking on it or by clicking `Alt+L` , as we can see below:



We can also set a default working directory to have all output files saved to, using the following command:

```
ERC --config SetWorkingDirectory C:\Users\htb-student\Desktop\
```

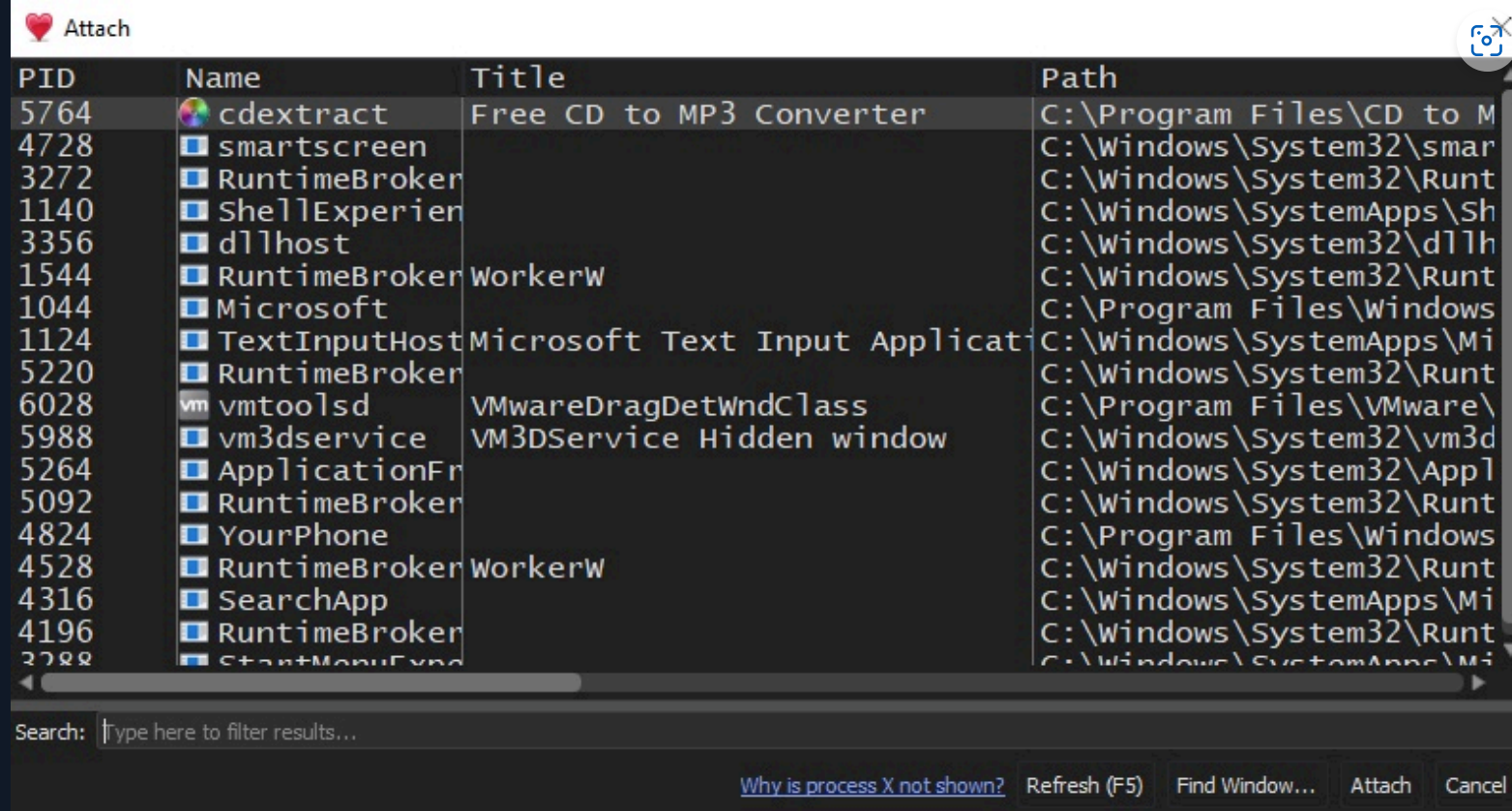
powershell

Now all of our output should be saved on our Desktop.

Debugging a Program

Whenever we want to debug a program, we can either run it through `x32dbg` , or run it separately and then attach to its process through `x32dbg` .

To open a program with `x32dbg` , we can select `File>Open` or press `F3` , which will prompt us to select the program to be debugged. If we wanted to attach to a process/program that is already running, we could select `File>Attach` or press `Alt+A` , and it will present us with various running processes accessible by our user:



We can select the process we want to debug and click on **Attach** to start debugging it.

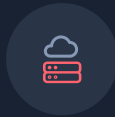
Tip: If we wanted to debug a process and it was not shown in the "Attach Window", we can try running x32dbg as an admin, by clicking on **File > Restart as Admin**, and then we will have access to all running processes on our VM.

Getting Help

If we wanted any help with either of these tools, we could refer to the [x64dbg documentation](#) and the [ERC documentation](#).

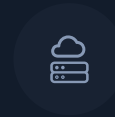
Now that we have all of our tools set up, we can start debugging our first software to try and find a stack overflow vulnerability and exploit it.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



VPN

Download your files and connect within your own environment



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

Target(s)

Spawn the target system to get IPs and answer questions

🔌 Spawn the target system



Enable step-by-step solutions ⓘ PRO

⚠️ Answer questions in English to ensure an accurate response.

Question 1

XP +40 ^

Try to RDP into the Windows VM using one of the methods mentioned above, and apply what you learned in this module. When you try to attach to 'Free CD to MP3 Converter', what is the name of its process?

RDP to with user "**htb-student**" and password "**Academy_student!**"

Write your answer

Submit

 Hint

← Previous

Section 2 / 11

 +20

Next

Mark Complete & Next